1 Funktionsumfang und Zielerreichung

Die Datenbank der Buchtausch-App bildet den Verleih privater Bücher in einer lokalen Gemeinschaft ab. Sie ist als Schema *buchtausch* in PostgreSQL 18 mit dreizehn Tabellen umgesetzt, zwölf Entitäten und einer Zuordnungstabelle. Tabelle 1 stellt die Anforderungen der Aufgabenstellung der Umsetzung gegenüber.

Anforderung	Umsetzung
Normalisiertes relationales Schema	13 Tabellen, dritte Normalform mit sechs ausgewiesenen Abweichungen (Abschnitt 5.2)
Bibliografische Buchdaten und Zustand	*Buchtitel* mit *Autor*, *Verlag*, *Genre* und *Sprache*, der Zustand am physischen *Exemplar*
Nutzerdaten	*Nutzer* und *Standort* mit Geokoordinaten
Ausleihdauer, Abholort, Zeitslots, Postversand	*MaxAusleihDauerTage*, *StandortID* und *VersandMöglich* am *Exemplar*, Abholfenster in *Zeitslot* ohne Überlappung, Ausleihdatum im Zeitslot
Räumliche Suche	Umkreissuche über die Koordinaten des *Standort*, Mittelpunkt und Radius als Parameter, Vorauswahl über einen Index
Ausleihvorgänge	*Ausleihe* mit Exemplar, Zeitslot, Ausleiher, Verleiher, Frist und Rückgabe, je Exemplar überlappungsfrei bis zur tatsächlichen Rückgabe
Bewertungen und Kommentare	*Bewertung* der Beteiligten einer Ausleihe nach der Rückgabe und *Buchbewertung* des Titels
Mindestens 10 Einträge je Tabelle	10 bis 16 Einträge je Tabelle, insgesamt 163 (Tabelle 3)
Testfälle und Dokumentation	95 Testfälle (51 negativ, 44 positiv), alle Statements der SQL-Datei kommentiert

Tabelle 1: Anforderungen der Aufgabenstellung und Umsetzung. Quelle: Eigene Darstellung

2 Datenmodell und Integritätsmechanismen

Das Modell trennt die bibliografischen Angaben (*Buchtitel*) vom physischen *Exemplar*, sodass ein Titel bei mehreren Besitzern erfasst wird, ohne diesen zu wiederholen. Die drei Dreifachbeziehungen sind als assoziative Entitäten umgesetzt, nämlich *Exemplar* (Buchtitel, Besitzer, Abholort), *Ausleihe* (Exemplar, Zeitslot, Ausleiher, Verleiher) und *Bewertung* (Ausleihe, Bewertender, Bewerteter). Die Regeln des Verleihs sind, im Rahmen der Möglichkeiten, Bedingungen der Datenbank (Tabelle 2).

Mechanismus	Anzahl	Wirkung
Primärschlüssel	13	Ein Identity-Schlüssel je Tabelle, zusammengesetzt nur in *Buchtitel_Autor*
Fremdschlüssel	19	17 mit *RESTRICT* zum Erhalt abgeschlossener Vorgänge, 2 mit *CASCADE* für Zeitslots und Autorenzuordnung, darunter 2 zusammengesetzte (Abholort gehört dem Besitzer, Zeitslot gehört zum Exemplar)
UNIQUE	9	E-Mail, ISBN, Verlagsname, Genre, Sprache, Bewertung je Ausleihe und Bewertendem, Titelbewertung je Nutzer, zwei Mehrspaltenschlüssel als Ziel der zusammengesetzten Fremdschlüssel
CHECK	14	Wertebereiche (Koordinaten, Erscheinungsjahr, Auflage, Zustand, Ausleihdauer, Sterne), Reihenfolge von Tagen, Uhrzeiten und Datumsangaben, Ausleiher ungleich Verleiher, Bewertender ungleich Bewerteter
EXCLUDE	2	Keine überlappenden Zeitslots und Verleihzeiträume je Exemplar, der Verleihzeitraum endet mit der tatsächlichen Rückgabe, über Bereichstypen und die Erweiterung *btree_gist* (PostgreSQL Global Development Group, 2026a, Kap. 5.5.6, 2026b, 2026c)
Trigger	2	Verleiher ist Besitzer, *RückgabeSpätestens* ergibt sich aus Ausleihdatum und *MaxAusleihDauerTage*, Ausleihdatum im Zeitslot, Bewertender und Bewerteter sind die Beteiligten der Ausleihe, Bewertung erst nach der Rückgabe

Tabelle 2: Integritätsmechanismen des Schemas (Anzahl laut SQL-Datei). Quelle: Eigene Darstellung

3 Metadaten der Datenbank

Die Datenbank umfasst 13 Tabellen mit zusammen 163 Einträgen. Die Zeilenzahlen stammen aus Abfrage 6 der SQL-Datei, die Testfälle aus dem Testprotokoll. Die Größe wird in Abschnitt 8 der SQL-Datei mit *pg_total_relation_size* je Tabelle einschließlich Indizes und mit *pg_database_size* für die gesamte Datenbank gemessen, die auch die Systemkataloge enthält und deshalb größer ist als die

Summe der Tabellen (PostgreSQL Global Development Group, 2026d, Kap. 9.28.7).

Tabelle	Einträge	Tests	Größe
Nutzer	12	7	40 kB
Standort	14	7	56 kB
Verlag	11	5	40 kB
Genre	10	4	40 kB
Sprache	10	4	40 kB
Autor	10	4	24 kB
Buchtitel	11	11	40 kB
Buchtitel_Autor	12	2	24 kB
Exemplar	14	8	40 kB
Zeitslot	15	11	48 kB
Ausleihe	15	16	64 kB
Bewertung	16	10	64 kB
Buchbewertung	13	6	48 kB
Summe Schema	**163**	**95**	**568 kB**
Datenbank gesamt			**11 MB**

Tabelle 3: Einträge, Testfälle und Größe je Tabelle (Größe einschließlich Indizes). Quelle: Eigene Darstellung

SQL-Dokumentation: Größe der Datenbank

```
-- Größe der gesamten Datenbank
SELECT pg_size_pretty(pg_database_size(
         current_database())) AS Datenbank;
-- Größe je Tabelle des Schemas buchtausch
-- mit Summe, inklusive Indizes
SELECT coalesce(c.relname::text, 'Summe')
         AS Tabelle,
       pg_size_pretty(sum(
         pg_total_relation_size(c.oid))) AS Größe
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE n.nspname = 'buchtausch' AND c.relkind = 'r'
GROUP BY ROLLUP (c.relname)
ORDER BY c.relname;
```

4 Abfragefunktionalität und Effizienz

Die sechs Abfragen der SQL-Datei decken die Funktionen der App ab. Abfrage 1 sucht nicht verliehene Exemplare eines Titels im Umkreis eines als Parameter übergebenen Standorts nach dem sphärischen Kosinussatz mit Vorauswahl über den Breitengrad (Veness, 2020, 2022, Grenzen in Abschnitt 5.5). Die Abfragen 2 bis 4 verbinden die drei Dreifachbeziehungen *Exemplar*, *Ausleihe* und *Bewertung* mit den Beteiligten. Abfrage 5 wertet die durchschnittliche Bewertung je Nutzer aus, Abfrage 6 zählt die Zeilen je Tabelle.

Fünf Indizes ergänzen die Indizes, die PostgreSQL für Primärschlüssel, Unique- und Ausschluss-Constraints selbst anlegt (PostgreSQL Global Development Group, 2026a, Kap. 5.5). *idx_standort_koordinaten* dient der Vorauswahl der Umkreissuche, die übrigen vier dienen als Join-Spalten der Abfragen. Abbildung 1 zeigt für die Abfrage aus Abschnitt 8 der SQL-Datei, dass der Planer für die Vorauswahl einen *Index Scan* über *idx_standort_koordinaten* wählt. Die Indexbedingung enthält allerdings nur den Breitengrad (Abschnitt 5.5).

SQL-Dokumentation: Plan der Vorauswahl

```
-- Plan der Vorauswahl der Umkreissuche
-- (Fenster um Berlin)
EXPLAIN SELECT StandortID
FROM buchtausch.Standort
WHERE Breitengrad BETWEEN 52.4 AND 52.6;
```

Plan

```
buchtausch=> EXPLAIN SELECT StandortID FROM buchtausch.Standort WHERE Breitengrad BETWEEN 52.4 AND 52.6;
                                  QUERY PLAN
-------------------------------------------------------------------------------------------
 Index Scan using idx_standort_koordinaten on standort  (cost=0.14..8.16 rows=1 width=4)
   Index Cond: ((breitengrad >= 52.4) AND (breitengrad <= 52.6))
(2 rows)
```

Abbildung 1: Plan der Vorauswahl der Umkreissuche (psql). Quelle: Eigene Darstellung

5 Rückmeldung, Präzisierung und Grenzen

5.1 Rückmeldung zu Phase 1 und 2

Die Empfehlungen zu Phase 1 (positive und negative Testfälle, Umfangsvorgaben) sind in Phase 2 durch 88 Testfälle, 21 Folien und eine einseitige Zusammenfassung umgesetzt. Phase 3 erweitert die Tests auf 95 (Tabelle 3). Die drei aufgebrachten Hinweise des Feedbacks zu Phase 2 sind in den Abschnitten 5.3 bis 5.5 adressiert.

5.2 Präzisierung der Normalisierung (Abweichung von Phase 1)

In Phase 1 wurden zwei transitive Abhängigkeiten übersehen. *Exemplar.BesitzerNutzerID* ist über *Standort.NutzerID* ableitbar, *Ausleihe.ExemplarID* über *Zeitslot.ExemplarID*. Beide Spalten bleiben erhalten, weil die Besitzbeziehung im ER-Modell eigenständig ist und der *EXCLUDE*-Constraint der *Ausleihe* nur eigene Spalten auswertet. Zusammengesetzte Fremdschlüssel verhindern Widersprü-

che. Mit den vier in Phase 1 begründeten Abweichungen, *Ausleihe.VerleiherNutzerID*, *Bewertung.BewerteterNutzerID*, *Ausleihe.RückgabeSpätestens* und Postleitzahl → Ort → Land in *Standort*, weicht das Modell an sechs Stellen von der dritten Normalform ab.

5.3 Trigger statt Constraints

Drei Regeln lesen Werte einer anderen Tabelle: Verleiher gleich Besitzer, *RückgabeSpätestens* aus *MaxAusleihDauerTage* und die Beteiligten einer Bewertung. Ein *CHECK*-Constraint darf nur die geprüfte Zeile verwenden. Für eine einmalige Prüfung gegen andere Zeilen beim Einfügen nennt die Dokumentation den Trigger (PostgreSQL Global Development Group, 2026a, Kap. 5.5.1). Die Besitzregel wäre als Fremdschlüssel (*ExemplarID*, *VerleiherNutzerID*) auf *Exemplar* möglich, würde aber dauerhaft gelten und einen Besitzerwechsel blockieren, solange Ausleihen verweisen. Der Trigger prüft nur beim Schreiben der Ausleihe und hält den Stand zum Zeitpunkt des Vorgangs fest, ebenso bei *RückgabeSpätestens*. Vorteile sind Regeln über Tabellengrenzen, ein fester Prüfzeitpunkt und eigene Fehlermeldungen. Nachteilig ist, dass ein Trigger nur auf seiner Tabelle wirkt und eine spätere Änderung des Ausleihers einer bewerteten Ausleihe nicht erkennt. Zudem ist die Regel prozedural, im Schema nicht als Constraint sichtbar und sperrt anders als ein Fremdschlüssel die gelesene Zeile nicht, sodass gleichzeitige Änderungen unerkannt bleiben können.

5.4 Nachbesserungen der Finalisierungsphase

Die Finalisierungsphase ergänzt die Trigger um zwei bisher der Anwendung überlassene Regeln. Eine Bewertung ist erst ab dem *Rückgabedatum* möglich, das Ausleihdatum muss im gewählten Zeitslot liegen. Zudem endete der Verleihzeitraum im *EXCLUDE*-Constraint stets mit *RückgabeSpätestens*, sodass ein vorzeitig zurückgegebenes Exemplar bis dahin gesperrt blieb, obwohl Abfrage 1 es als verfügbar anzeigte. Er endet nun mit *LEAST(Rückgabedatum, RückgabeSpätestens)*. Sieben neue Testfälle belegen dies. Zwei Lücken bleiben. Wird das *Rückgabedatum* nach einer Bewertung geändert, erkennt der Trigger auf *Bewertung* dies nicht (Abschnitt 5.3), und eine überfällige, nicht zurückgegebene Ausleihe sperrt das Exemplar nur bis *RückgabeSpätestens*.

5.5 Grenzen der Umkreissuche

Mittelpunkt und Radius sind seit der Finalisierungsphase Parameter. Bei den Vorgaben Standort 3 und 300 km liegt Hamburg mit 254,7 km im Umkreis, München mit 506,7 km nicht. Die Vorauswahl begrenzt nur den Breitengrad, das Fenster $\lambda \pm (d/R)/\cos\varphi$ für den Längengrad fehlt (Veness, 2020). Der Kosinussatz liefert die Luftlinie statt der Wegstrecke und kann auch bei identischen Koordinaten abbrechen, weil das Argument von *acos* aufgrund Rundung über 1 liegen kann. Die Zeitslots gehen in die Verfügbarkeit nicht ein. Verbessern ließe sich dies durch ein Fenster auch für den Längengrad und die Prüfung der Zeitslots. Wegstrecken und höhere Genauigkeit erfordern PostGIS.

6 Repository und Reproduktion

Quelltext, Berichte der drei Phasen und eine README-Datei mit Installationsanleitung liegen im GitHub-Repository `https://github.com/velvet-orbit-canyon-lunar/sql-data-mart/`

Literaturverzeichnis

PostgreSQL Global Development Group. (14. Mai 2026a). *5.5. Constraints*. PostgreSQL 18 Documentation. `https://www.postgresql.org/docs/18/ddl-constraints.html`

PostgreSQL Global Development Group. (14. Mai 2026b). *8.17. Range types*. PostgreSQL 18 Documentation. `https://www.postgresql.org/docs/18/rangetypes.html`

PostgreSQL Global Development Group. (14. Mai 2026c). *F.8. btree_gist — GiST operator classes with B-tree behavior*. PostgreSQL 18 Documentation. `https://www.postgresql.org/docs/18/btree-gist.html`

PostgreSQL Global Development Group. (14. Mai 2026d). *9.28. System administration functions*. PostgreSQL 18 Documentation. `https://www.postgresql.org/docs/18/functions-admin.html`

Veness, C. (2020). *Selecting points from a database by latitude/longitude within a bounding circle*. Movable Type Scripts. `https://www.movable-type.co.uk/scripts/latlong-db.html`

Veness, C. (2022). *Calculate distance, bearing and more between latitude/longitude points*. Movable Type Scripts. `https://www.movable-type.co.uk/scripts/latlong.html`